

START HERE

Autonomous Quadcopter Drone Interceptor

Dor Varsulker — Workshop in Autonomous Systems Simulation, BSc Computer Science

What this project is

A simulation of an autonomous counter-drone interceptor: a 1 kg quadcopter that must detect, track and physically collide with a second, possibly evasive, flying target. Nothing physical is built — the airframe, sensor, wind and threat all live inside the MuJoCo physics engine, driven by a Python guidance, navigation and control stack.

The interceptor is given only what a real system would have: range and angle readings that are noisy, biased, sampled at a finite rate and delivered late. It must estimate the target's true motion from them, decide how to accelerate to hit it, and translate that decision down to four motor speeds — while respecting real physical limits, since a quadcopter can only accelerate sideways by tilting, and tilting takes time. Every stage is explicit, classical mathematics rather than a learned model: deep reinforcement learning was evaluated and deliberately rejected in favour of **determinism** (identical seed and configuration give a byte-identical run log), **valid physics** and **explainability**.

At a glance

	Result	
Mission success rate	95% over 100 seeded randomized 3D trials	✓
Max target speed intercepted	89.7 km/h (requirement: 83.6 km/h)	✓
Median miss distance	0.024 m (a hit is anything within 1.05 m)	✓
Named scenarios	11/11 baseline pass every KPI; 11/11 stress probes intercept	✓
Command saturation	77% compliance — reported as failing, not tuned away	⚠
Test suite	221 headless, seeded tests, ~80 s	✓

Measured on the canonical batch: `--trials 100 --seed 0`. Full numbers in the project report.

Where to read more

This document gets you oriented and running. Everything else lives in five documents:

Document	Answers
Project Report (the main submission PDF)	The full narrative: rationale, architecture, methodology, results and limitations.
<code>docs/Autonomous_Drone_Interceptor_Design_Review.md</code>	Why this architecture and this guidance law were chosen.
<code>docs/PROJECT_EXPLAINED.md</code>	How every stage works from scratch, no control-theory background assumed; has a glossary.
<code>docs/ENGINEERING_STANDARDS.md</code>	The rules the code is built to; source comments cite it by name.
<code>docs/RESULTS.md</code>	Measured KPIs, breakdowns, known limits and the commands that regenerate them.

How it works, in one page

The interceptor's decision-making is a chain of six stages. Each consumes only the published output of its immediate predecessor, and the chain closes back into the physics:

```
[1 SIMULATION] -- SensorMeasurement (noisy, delayed) -->
[2 ESTIMATION] -- TargetEstimate (relative state + cov.) -->
[3 GUIDANCE]   -- AccelerationCommand (ideal) -->
[4 LIMITER]    -- LimitedAcceleration (physically safe) -->
[5a OUTER CTL] -- AttitudeReference (roll/pitch/yaw+thr.) -->
[5b INNER CTL] -- BodyWrench (torques + thrust) -->
[6 MIXER]      -- MotorCommand (4 x rotor RPM) --> back into [1]
```

Stage	What it does	Lives in
1 — Simulation	MuJoCo physics, rotor model, the noisy delayed sensor, target trajectories and wind.	simulation/
2 — Estimation	An Extended Kalman Filter recovering a clean, latency-compensated picture of the target's relative motion.	estimation/
3 — Guidance	The Optimal Guidance Law: a lag-aware Zero-Effort-Miss law that leads the target instead of chasing it.	guidance/
4 — Limiter	The single place acceleration is clamped to what the airframe can produce — and the single place saturation is measured.	control/
5 — Flight control	A 50 Hz outer loop mapping acceleration to tilt and thrust, and a 400 Hz inner attitude loop realising it.	control/
6 — Mixer	Turns thrust and torques into four rotor speeds, clamped to real motor limits.	control/

Three rules make the results trustworthy, and are worth knowing before reading any output. **No stage may reach around its input** — guidance cannot read raw sensor data, and nothing outside the simulation may read ground truth. **Bad data stops the run** — a NaN in any message, filter divergence, or an unknown scenario key raises immediately rather than propagating. And **all randomness flows through one seeded factory**, so a run is reproducible from its seed and every run records that seed, its resolved parameters and the git hash alongside its log.

Saturation warnings are not errors. Lines beginning command saturation: during a run are the limiter honestly reporting that guidance asked for more acceleration than the airframe can produce. Measuring that is a graded KPI, which is why it is printed rather than swallowed.

Getting it running

You need Python 3.11 or newer, about 1 GB of disk and CPU only — no GPU, no CUDA and no separate MuJoCo installation, since the pip wheel bundles its own native libraries on Linux, macOS and Windows. A display is needed only for the replay viewer; everything else is headless. The whole sequence below takes roughly fifteen minutes, most of which is the machine running.

1. Install

```
python -m venv .venv
source .venv/bin/activate          # Windows: .\.venv\Scripts\Activate.ps1
pip install -e ".[dev]"            # pinned runtime deps + pytest and ruff
```

The editable install puts the interceptor package on the path so imports work from anywhere. Versions are pinned deliberately, because determinism is the project's core guarantee.

2. Verifv the environment

```
python scripts/check_env.py
```

This is the environment doctor: it checks the Python version, imports every dependency, reports the MuJoCo version and GL backend, steps a trivial physics model and renders one frame off-screen. Exit code 0 means you are ready. The render check is the only step with a host-specific dependency — if it fails, set `MUJOCO_GL=egl` (headless GPU) or `MUJOCO_GL=osmesa` (pure software), or simply skip those tests with `pytest -m "not mujoco"`. No simulation result depends on rendering.

3. Flv your first interception

```
$ python scripts/run_intercept.py --target 8 3 6 --seconds 9
Ran 2048 steps headlessly against target [8.0, 3.0, 6.0].
  min miss distance: 0.037 m at t=5.12 s  [HIT vs R_miss <= 1.05 m]
  run log:  results/intercept/run_log.csv
  snapshot: results/intercept/run_config.json
```

About five seconds of wall clock. It flies the complete pipeline — MuJoCo physics, the noisy delayed sensor, the EKF, the guidance law, the limiter, the dual control loop and the mixer — against a target hovering at (8, 3, 6) m. Useful flags: `--start X Y Z` (interceptor start, default 0 0 2), `--seed N`, `--seconds S` (an upper bound; the run stops at closest approach), `--no-terminate`, `--run-id NAME`, `--params FILE.yaml`.

4. Run the tests

```
pytest                # 221 tests, ~80 s
pytest -m "not mujoco" # skip the tests needing an off-screen GL context
pytest tests/unit      # fast component tests only
ruff check src tests scripts
```

Reproducing the results

The results/ directory is generated output: git-ignored and **not included in the submission**, so you generate your own runs. These three commands regenerate every published number, and everything is seeded, so what you generate is what the report describes.

```
# 11 static and linear geometries, with a KPI table and per-scenario plots (~21 s)
python scripts/run_scenarios.py scenarios/ --report --results-dir results/scenarios

# 11 stress probes: evasive weaves, 84-90 km/h ramps, wind and gusts (~14 s)
python scripts/run_scenarios.py scenarios/stress --results-dir results/stress

# The headline number: 100 randomized 3D trials, seeded (~1 min 50 s)
python scripts/run_montecarlo.py --trials 100 --seed 0 --report \
    --results-dir results/montecarlo
```

The scenario runner reads a directory *without* recursing, hence two suite commands; --trials 10 --seed 0 gives a ten-second look, though the headline rates need the full 100. Each run writes run_log.csv and run_config.json (seed, resolved parameters, git hash) into its own directory; suites and batches add a KPI summary, plots and a manifest. A hit is any miss within 1.05 m, but a scenario can show a green miss and still fail overall: six metrics are graded, and saturation is the strict one.

Watching a run

Nothing can be replayed until you have flown something, since no runs ship with the project. Generate first, then replay — the quickest path is a single interception.

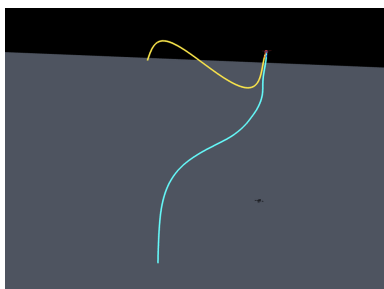
```
# 1. generate a run -- any of these
python scripts/run_intercept.py --target 8 3 6 --seconds 9
    -> results/intercept/                one run, ~5 s
python scripts/run_scenarios.py scenarios/stress --results-dir results/stress
    -> results/stress/<scenario>/        one directory per scenario
python scripts/run_montecarlo.py --trials 10 --seed 0 --report \
    --results-dir results/montecarlo
    -> results/montecarlo/trial_XXXX/    one directory per trial

# 2. replay any run directory you just created
python scripts/replay.py results/<target_run_dir>

python scripts/replay.py results/intercept                # top isometric view
python scripts/replay.py results/stress/sinusoidal_3d_spiral # a stress scenario
python scripts/replay.py results/montecarlo/trial_0007     # one Monte-Carlo trial

--view interceptor    chase camera    --speed 0.5    slow motion
--loop               replay continuously
```

Point the viewer at a *leaf* run directory — one holding run_log.csv — not at a suite root such as results/stress, which holds only per-scenario folders.



The replay viewer showing the sinusoidal_3d_spiral stress scenario: interceptor trail in cyan, target in yellow. Without --loop it freezes on the intercept frame. The viewer re-renders the recorded log rather than re-simulating, so it cannot change any result — which is why it is the one interactive tool in an otherwise headless project.

Where things live

```
Autonomous-Quadcopter-Drone-Interceptor/
|- docs/           Design review, PROJECT_EXPLAINED, standards, results, guide
|- models/         MuJoCo world: scene.xml, quadcopter.xml, target.xml
|- scenarios/       11 baseline YAML + ablation/ + stress/ (11 probes)
|- scripts/        check_env, run_intercept, run_scenarios, run_montecarlo,
|                  run_stub_pipeline, replay
|- src/interceptor/
|   |- config/      constants.py (physical) + params.py (tunable)
|   |- common/      types, frames, rng, logging, guards
|   |- simulation/  [1] plant, actuators, sensors, trajectories, wind
|   |- estimation/  [2] ekf
|   |- guidance/    [3] ogl, zem, time_to_go
|   |- control/     [4,5] command_limiter, outer_loop, inner_loop, motor_mixer
|   |- pipeline/    [6] orchestrator, scheduler
|   +- analysis/    [5] kpis, scenarios, montecarlo, reporting
|- tests/          unit/ + integration/ -- headless, seeded (221 tests)
+- results/        Generated logs, snapshots, reports (git-ignored)
```

Writing your own scenario

Drop a YAML file anywhere and point the runner at it. An optional params: block deep-merges onto the defaults, so any tunable (sensor noise, EKF, guidance, limiter, PID) can be overridden for one scenario without touching code.

```
name: my_scenario
seed: 0
target_class: moving      # "static" | "moving" -- selects the time KPI bound
guidance_law: OGL         # OGL is the sole law; anything else is rejected
time_limit_s: 20.0
wind_preset: moderate     # optional: calm | moderate | gusty
interceptor:
  start_m: [0.0, 0.0, 2.0]
target:
  type: linear             # static | linear | sinusoidal | varying_speed
  start_position_m: [9.0, -5.0, 5.0]
  velocity_m_s: [0.0, 2.0, 0.0]

$ python scripts/run_scenarios.py my_scenario.yaml
```

type	Required target keys
static	position_m
linear	start_position_m, velocity_m_s
sinusoidal	start_position_m, drift_velocity_m_s, amplitude_m, frequency_hz
varying_speed	start_position_m, heading, initial_speed_m_s, peak_speed_m_s, ramp_duration_s

The loader fails loud: unknown keys, missing keys, a bad vector shape, a guidance law other than OGL, or setting both wind_preset and an explicit params.wind block will all error out immediately, naming the offending section, rather than silently picking a default.

Next stop: the Project Report PDF for the full picture — why this architecture, how each stage works, the complete measured results, and the three known limitations. For the ground-up explanation of the mathematics, read docs/PROJECT_EXPLAINED.md.